12

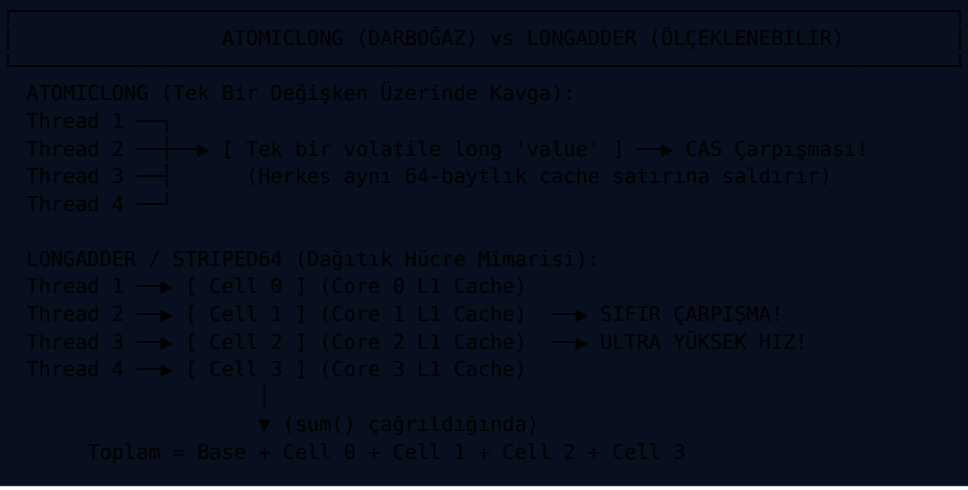
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Yüksek Ölçekli Sayaçlar: LongAdder vs AtomicLong (Striped64 & Dağıtık Hücreler)

High-Throughput Scalability: LongAdder vs AtomicLong (Striped64 & Cell Arrays)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir süpermarkette tek bir kasa olduğunu düşünün (AtomicLong). 50 müşteri aynı anda tek bir kasiyere para ödemeye çalışırsa birbirini ezer, kuyruk uzar. LongAdder ise süpermarkete 8 ayrı kasa açmak gibidir! Müşteriler farklı kasalara (Cell) dağılır, kimse kimseyi beklemeden parasını öder. Günün sonunda patron kasaların hepsini toplar (sum()) ve toplam ciroyu bulur! Okuma nadir, yazma çoksa LongAdder 100 kat daha hızlıdır!

🇬🇧 ENGLISH (ELI5)

Picture a stadium with a single concession register (AtomicLong). Thousands of fans violently rush that lone cashier simultaneously, resulting in massive bottlenecks and dropped orders. LongAdder opens 16 parallel registers distributed across the stadium (Cell array). Fans transact at their nearest register with zero friction. At halftime, the accountant aggregates balances across all registers (`sum()`). When writes dominate, LongAdder outpaces AtomicLong by orders of magnitude!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Striped64 Mimarisi ve LongAdder Derinliği: 1) Çekişme Azaltma (Contention Mitigation): `AtomicLong` tek bir bellek adresi üzerinde CAS yapar; thread sayısı arttıkça L1 cache geçersiz kılma (cache invalidation) fırtınası kopar. 2) `Cell[]` Dizisi: LongAdder, Doug Lea tarafından yazılan `Striped64` sınıfından türer. Çekişme tespit edildiğinde thread ID'sinin hash'ine göre dinamik olarak `Cell` dizisi büyütülür (2, 4, 8, 16...). Her `Cell` kendi içinde `\@Contended` anotasyonu ile 128 bayt doldurulmuştur (Padding), böylece False Sharing tamamen önlenir! 3) Ne Zaman AtomicLong, Ne Zaman LongAdder?: Eğer atomik bir sıra numarası (Sequence ID: `incrementAndGet()`) gerekiyorsa `AtomicLong` zorunludur. Eğer sadece metrik toplama, sayaç tutma veya istatistik yapılıyorsa kesinlikle `LongAdder` tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Internal Mechanics of Striped64: 1) The Contention Bottleneck of AtomicLong: AtomicLong forces all concurrent cores to serialize writes to a single memory offset, saturating the cache bus with MESI invalidation traffic. 2) Striped64 Cell Dynamism: LongAdder maintains a `base` variable. If a CAS on `base` fails due to contention, it allocates a `Cell[]` table sized to a power of two (capped at CPU core count). Each thread hashes into a specific `Cell`. Crucially, each `Cell` is annotated with `\@jdk.internal.vm.annotation.Contended`, enforcing 128-byte cache-line isolation to eliminate false sharing. 3) Architectural Trade-Off: `LongAdder.sum()` is an eventual snapshot, not an atomic instantaneous freeze. Use `AtomicLong` when sequential order or exact compare-and-set guards are required (`compareAndSet()`); use `LongAdder` for high-frequency telemetry, request counting, and metrics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Yüksek Ölçekli Metrik Toplama için LongAdder
import java.util.concurrent.atomic.LongAdder;

public class MetricsCollector {
    // Saniyede 1 milyon HTTP isteğini kaydetmek için idealdir
    private final LongAdder requestCount = new LongAdder();

    public void onRequestReceived() {
        requestCount.increment(); // Çarpışmasız, dağıtık hücreye yazar
    }

    public long getTotalRequests() {
        return requestCount.sum(); // Tüm hücreleri toplar
    }

    void main() throws InterruptedException {
        var metrics = new MetricsCollector();
        int threads = 8;
        int reqPerThread = 500_000;

        Thread[] workers = new Thread[threads];
        for (int i = 0; i < threads; i++) {
            workers[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < reqPerThread; j++) {
                    metrics.onRequestReceived();
                }
            });
        }

        for (Thread t : workers) t.join();

        IO.println("Toplam Başarılı İstek: " + metrics.getTotalRequests()); // 4,000,000
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Banka hesap bakiyesi veya sıra numarası için LongAdder kullanabilir miyiz?' Yanıt: 'HAYIR! LongAdder atomik bir `compareAndSet` veya anlık kesin okuma sunmaz. `sum()` çağrısı yapılırken arka planda yazmalar devam edebilir. Sıra numarası için daima AtomicLong kullanılmalıdır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'Can we use LongAdder to issue unique order IDs or manage account balances?' Answer: 'NO! LongAdder does not provide an atomic `compareAndSet()` or transactional read-and-mutate guarantee. `sum()` yields an eventual aggregate, not an isolated lockstep snapshot. Unique sequences require AtomicLong.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LongAdder, yüksek çekişmeli yazma işlemlerinde AtomicLong'dan 10-100 kat daha hızlıdır.
- Striped64 hücreleri \@Contended ile pad edildiği için False Sharing yaşanmaz.
- Benzersiz sıra numarası (Sequence ID) için AtomicLong; metrikler için LongAdder seçilir.

🎯 Key Takeaways for the Interview

- LongAdder achieves 10-100x throughput over AtomicLong under high write contention.
- Striped64 cells are padded to prevent False Sharing across CPU cache lines.
- Use AtomicLong for atomic sequential IDs; use LongAdder for high-volume metrics.